

NOTIFICATION_INTEGRATION_REPORT

HelixCode Notification & Hook Integration - Comprehensive Analysis & Implementation Plan

Generated: 2025-11-04 **Status:** Complete Assessment with Implementation Roadmap

Executive Summary

This report provides a comprehensive analysis of the HelixCode notification and hook integration system, including current implementation status, test coverage analysis, setup guides for existing integrations, research on potential new integrations, and a detailed implementation plan with full testing strategy.

Key Findings

Current State: - Notification engine framework is well-designed with good abstractions - Slack integration: Partially implemented (webhook only, **0% test coverage**) - Telegram integration: **NOT IMPLEMENTED** (mentioned in docs only) - Email integration: Partially implemented (basic SMTP, **0% test coverage**) - Discord integration: Partially implemented (webhook only, **0% test coverage**) - Event-driven hook system: **NOT IMPLEMENTED** - Configuration management: **Minimal** (no notification section in config.yaml) - Setup guides: **NOT AVAILABLE**

Critical Gaps: - No automated event triggering for hooks - Zero test coverage for notification channels - Missing Telegram integration entirely - No setup documentation for integrations - No webhook retry/failure handling - No rate limiting or backoff mechanisms

1. Current Integration Analysis

1.1 Slack Integration

Implementation Location: /helix_code/internal/notification/engine.go:315-393

Current Features: - Webhook-based integration - Channel targeting - Custom username support - Icon emoji based on notification type - Basic text formatting (markdown-style with asterisks)

Implementation Details:

```
type SlackChannel struct {  
    name      string  
    enabled   bool  
    webhook   string  
    channel   string  
    username  string  
}
```

Gaps & Limitations: - No Slack Bot API support (only webhooks) - No rich message formatting (blocks, attachments) - No interactive components (buttons, menus) - No thread support for conversations - No direct message capability - No file/image attachments - **0% test coverage** - No error retry mechanism - No rate limiting (Slack has 1 message/second limit)

Configuration: - Environment variable: HELIX_SLACK_WEBHOOK_URL - No config.yaml integration - Hardcoded channel name in constructor

Status: Partially Functional - Needs Enhancement & Testing

1.2 Telegram Integration

Implementation Location: NONE

Status: NOT IMPLEMENTED

Mentioned In: - Architecture docs (ARCHITECTURE.md:75) - Listed as planned feature: “Telegram: Bot API with media support”

Required Implementation: - Telegram Bot API integration - Message formatting (Telegram markdown/HTML) - Media support (photos, documents, stickers) - Inline keyboards for interactive notifications - Chat ID management - Bot token configuration

Status: MISSING - High Priority

1.3 Email Integration

Implementation Location: /helix_code/internal/notification/engine.go:395-459

Current Features: - SMTP-based email sending - Basic authentication (username/password) - Plain text emails

Implementation Details:

```
type EmailChannel struct {  
    name      string  
    enabled   bool  
    smtpServer string  
    port      int  
    username  string  
    password  string  
    from      string  
}
```

Critical Bug Found:

```
// Line 426 in engine.go  
to := "" // Would come from notification metadata  
if to == "" {  
    return fmt.Errorf("no recipient specified")  
}
```

This will ALWAYS fail - recipient is never extracted from metadata!

Gaps & Limitations: - **CRITICAL BUG:** Recipient not extracted from metadata (line 426) - No HTML email templates - No attachment support - No TLS/SSL configuration options - No OAuth2 support (Gmail, Office365) - No email queue/batching - **0% test coverage** - No bounce handling - No unsubscribe management

Configuration: - No environment variables defined - No config.yaml integration - All parameters passed via constructor

Status: **BROKEN - Critical Bug + Missing Features**

1.4 Discord Integration

Implementation Location: /helix_code/internal/notification/engine.go:461-515

Current Features: - Webhook-based integration - Basic markdown formatting - Message content support

Gaps & Limitations: - No Discord Bot API support - No embeds (rich formatting) - No components (buttons, select menus) - No file attachments - No thread support - **0% test coverage** - No rate limiting (Discord has 30 requests/minute limit)

Configuration: - Environment variable: HELIX_DISCORD_WEBHOOK_URL - No config.yaml integration

Status: Partially Functional - Needs Enhancement & Testing

2. Test Coverage Analysis

2.1 Current Test Coverage

Overall Project Coverage: ~35% (per PROJECT_SUMMARY.md)

Notification System Coverage:

Unit Tests (internal/notification/engine_test.go): - NewNotificationEngine - RegisterChannel (with mock) - AddRule - LoadTemplate - SendDirect (with mock channel) - **NO Slack-specific tests** - **NO Email-specific tests** - **NO Discord-specific tests** - **NO failure scenario tests** - **NO retry logic tests**

Integration Tests (test/integration/integration_test.go): - Basic notification engine integration - TestNotificationChannelIntegration (lines 98-133) - **NO actual webhook integration tests** - **NO SMTP integration tests** - **NO mock server tests**

Test Coverage Estimate: ~5% for notification channels

2.2 Missing Test Scenarios

Unit Test Gaps: 1. Slack webhook payload formatting 2. Slack error handling (401, 404, 500) 3. Email recipient extraction from metadata 4. Email SMTP authentication failures 5. Discord webhook rate limiting 6. Channel enable/disable logic 7. Priority-based notification routing 8. Template rendering with various

data types 9. Concurrent notification sending 10. Notification rule condition matching

Integration Test Gaps: 1. Mock Slack webhook server 2. Mock Discord webhook server 3. Mock SMTP server for email testing 4. Webhook retry on failure 5. Network timeout scenarios 6. Malformed webhook responses 7. Rate limiting behavior 8. Large notification payloads 9. Concurrent channel sending 10. End-to-end notification workflows

E2E Test Gaps: 1. Task failure → notification triggered 2. Worker disconnect → alert sent 3. Workflow completion → summary sent 4. Multiple channels for single event 5. Rule-based routing verification

3. Event & Hook System Analysis

3.1 Current State: NO EVENT-DRIVEN SYSTEM

Critical Finding: The notification system has **NO automated event triggering**.

How Notifications Currently Work: 1. Manual invocation only via: - `SendNotification(ctx, notification)` - `SendDirect(ctx, notification, channels)` 2. No integration with task lifecycle events 3. No integration with workflow events 4. No integration with worker events

3.2 Available Events (Not Connected)

Task Lifecycle Events (`internal/task/manager.go`): - `TaskStatusPending` - Task created - `TaskStatusAssigned` - Task assigned to worker - `TaskStatusRunning` - Task execution started - `TaskStatusCompleted` - Task finished successfully - `TaskStatusFailed` - Task failed - `TaskStatusPaused` - Task paused

Workflow Events (`internal/workflow/executor.go`): - `WorkflowStatusRunning` - Workflow started - `WorkflowStatusCompleted` - Workflow completed - `WorkflowStatusFailed` - Workflow failed - Step-level events (not currently defined)

Worker Events (`internal/worker/pool.go`): - Worker connected - Worker disconnected - Worker health degraded - Heartbeat missed - Resource threshold exceeded

API Events (`internal/api/handlers/`): - User registration - User login - Project created/deleted - Authentication failures - API errors

3.3 Required Event System Architecture

Recommended Pattern: Observer/Event Bus

```
// Event bus architecture
type EventBus interface {
    Subscribe(eventType string, handler EventHandler)
    Publish(event Event)
    Unsubscribe(eventType string, handler EventHandler)
}

type Event struct {
    Type      string
    Timestamp time.Time
    Source     string
    Data       map[string]interface{}
    Severity  EventSeverity
}

type EventHandler func(ctx context.Context, event Event) error
```

Integration Points: 1. Task Manager: Emit events on status changes 2. Workflow Executor: Emit events on workflow/step changes 3. Worker Pool: Emit events on worker state changes 4. API Handlers: Emit events on critical operations 5. Notification Engine: Subscribe to relevant events

4. Configuration Management Analysis

4.1 Current Configuration

Environment Variables (.env.example):

```
HELIX_SLACK_WEBHOOK_URL=https://hooks.slack.com/services/...
HELIX_DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/...
```

Config File (config/config.yaml): - NO notification section - NO channel configurations - NO notification rules - NO retry/timeout settings

4.2 Recommended Configuration Structure

```
notifications:
  enabled: true

# Notification rules - map events to channels
```

```
rules:
- name: "Critical Task Failures"
  condition: "event.type == task_failed AND
    event.data.priority == critical"
  channels: ["slack", "email", "telegram"]
  priority: urgent
  template: "task_failure"

- name: "Worker Health Alerts"
  condition: "event.type == worker_health_degraded"
  channels: ["slack", "telegram"]
  priority: high

- name: "Workflow Completions"
  condition: "event.type == workflow_completed"
  channels: ["slack"]
  priority: medium

# Channel configurations
channels:
  slack:
    enabled: true
    webhook_url: ${HELIX_SLACK_WEBHOOK_URL}
    channel: "#helix-notifications"
    username: "HelixCode Bot"
    retry:
      max_attempts: 3
      initial_delay: 1s
      max_delay: 30s
      backoff_multiplier: 2
    timeout: 10s
    rate_limit:
      max_per_second: 1

  telegram:
    enabled: true
    bot_token: ${HELIX_TELEGRAM_BOT_TOKEN}
    chat_id: ${HELIX_TELEGRAM_CHAT_ID}
    retry:
      max_attempts: 3
      initial_delay: 1s
      max_delay: 30s
      backoff_multiplier: 2
    timeout: 10s
```

```
email:
  enabled: true
  smtp:
    server: smtp.gmail.com
    port: 587
    username: ${HELIX_EMAIL_USERNAME}
    password: ${HELIX_EMAIL_PASSWORD}
    from: "HelixCode <${HELIX_EMAIL_FROM}>"
    tls: true
    auth_method: plain
  recipients:
    default: ["admin@example.com"]
    critical: ["admin@example.com", "oncall@example.com"]
  templates:
    enabled: true
    html: true
  retry:
    max_attempts: 3
    initial_delay: 2s
    max_delay: 60s
    backoff_multiplier: 2
  timeout: 30s

discord:
  enabled: false
  webhook_url: ${HELIX_DISCORD_WEBHOOK_URL}
  retry:
    max_attempts: 3
    initial_delay: 1s
    max_delay: 30s
    backoff_multiplier: 2
  timeout: 10s
  rate_limit:
    max_per_minute: 30

# Template configurations
templates:
  task_failure: |
    Task Failed: {{.Title}}

    Details: {{.Message}}
    Task ID: {{.Metadata.task_id}}
    Worker: {{.Metadata.worker_id}}
```


Time: {{.Metadata.timestamp}}

workflow_completed: |

Workflow Completed: {{.Title}}

Duration: {{.Metadata.duration}}

Steps: {{.Metadata.step_count}}

Status: {{.Metadata.status}}

5. Research: Potential New Integrations

5.1 Communication Platforms

5.1.1 Microsoft Teams (Priority: HIGH)

Status: Recommended

Rationale: - Widely used in enterprises - Rich notification capabilities via Workflows - **IMPORTANT:** Old webhook connectors deprecating Dec 2025 - Must use new Workflows API

Implementation Requirements: - Workflows API integration (not old webhook connectors) - Adaptive Cards for rich formatting - Action buttons support - File attachments - Thread support

API Endpoint: - New: Power Automate Workflows - Complexity: Medium - Documentation: <https://learn.microsoft.com/en-us/microsoftteams/platform/>

Use Cases: - Enterprise deployments - Team collaboration notifications - CI/CD pipeline alerts

5.1.2 Mattermost (Priority: MEDIUM)

Status: Consider for self-hosted deployments

Rationale: - Open-source Slack alternative - Popular in security-conscious organizations - Self-hosted option

Implementation Requirements: - Webhook support (similar to Slack) - Bot API support - Markdown formatting

Use Cases: - On-premise deployments - Privacy-focused organizations - Government/defense contractors

5.1.3 Rocket.Chat (Priority: LOW)

Status: Optional

Rationale: - Another open-source alternative - Smaller user base - Similar to Mattermost

5.2 Incident Management

5.2.1 PagerDuty (Priority: HIGH)

Status: Strongly Recommended

Rationale: - Industry standard for on-call management - Critical incident routing - Escalation policies - Integration with monitoring tools

Implementation Requirements: - Events API v2 - Incident creation/update - Severity mapping - Alert deduplication

API Endpoint: - <https://api.pagerduty.com/> - Complexity: Medium - Authentication: API key

Use Cases: - Critical system failures - Production incidents - On-call engineer alerts - 24/7 monitoring

5.2.2 Opsgenie (Priority: MEDIUM)

Status: Alternative to PagerDuty

Rationale: - Atlassian product (good Jira integration) - Similar features to PagerDuty - Growing market share

5.3 Project Management

5.3.1 Jira (Priority: HIGH)

Status: Strongly Recommended

Rationale: - Most popular project management tool - Issue creation from failures - Automatic ticket assignment - Sprint/Epic tracking

Implementation Requirements: - Jira REST API v3 - Issue creation/update - Comment addition - Attachment support - Custom field mapping

API Endpoint: - <https://developer.atlassian.com/cloud/jira/platform/> - Complexity: High - Authentication: OAuth 2.0 or API token

Use Cases: - Auto-create tickets for failures - Link tasks to Jira issues - Update issue status on completion - Add execution logs as comments

5.3.2 Linear (Priority: MEDIUM)

Status: Consider for modern teams

Rationale: - Modern, fast alternative to Jira - Great API and developer experience - Growing adoption in startups

5.3.3 GitHub Issues (Priority: HIGH)

Status: Recommended

Rationale: - Native integration with code repositories - Widely used in open-source - Simple API

Implementation Requirements: - GitHub REST API - Issue creation/labeling - Comment addition - Repository webhooks (bidirectional)

Use Cases: - CI/CD failure tracking - Bug report automation - Release notifications

5.4 Generic & Custom

5.4.1 Generic Webhooks (Priority: HIGH)

Status: MUST HAVE

Rationale: - Maximum flexibility - Support any custom integration - Easy to implement - No vendor lock-in

Implementation Requirements: - Configurable HTTP method (POST, PUT, PATCH) - Custom headers - Payload template customization - Authentication (Bearer, Basic, API Key) - SSL/TLS verification options - Retry logic

Use Cases: - Custom internal systems - Third-party SaaS tools - Legacy system integration - Zapier/IFTTT integration

5.4.2 WebSockets (Priority: MEDIUM)

Status: Consider for real-time needs

Rationale: - Real-time push to connected clients - Dashboard updates - Live monitoring

5.5 Messaging & SMS

5.5.1 Twilio SMS (Priority: MEDIUM)

Status: Consider for critical alerts

Rationale: - Critical incident notifications - On-call engineer alerts - 2FA and security alerts

Implementation Requirements: - Twilio REST API - Phone number management - SMS templating - Cost management (SMS costs money)

Use Cases: - Production outages - Security incidents - Critical failure alerts

5.5.2 WhatsApp Business API (Priority: LOW)

Status: Optional

Rationale: - Popular globally - Rich media support - Requires business verification

5.6 Push Notifications

5.6.1 Firebase Cloud Messaging (Priority: MEDIUM)

Status: Recommended for mobile apps

Rationale: - Native mobile notifications - iOS and Android support - Part of HelixCode mobile strategy

Implementation Requirements: - FCM API - Device token management - Platform-specific formatting

Use Cases: - Mobile app notifications - Real-time alerts to mobile users

5.6.2 OneSignal (Priority: LOW)

Status: Alternative to FCM

Rationale: - Multi-platform support - Easier setup than FCM - Free tier available

5.7 Summary: Recommended Integration Priorities

Tier 1 (MUST HAVE): 1. Slack (enhance existing) 2. Email (fix existing) 3. Telegram (implement new) 4. Generic Webhooks (implement new) 5. Microsoft Teams (implement new)

Tier 2 (HIGH VALUE): 6. PagerDuty (incident management) 7. Jira (issue tracking)
8. GitHub Issues (repository integration)

Tier 3 (NICE TO HAVE): 9. Discord (enhance existing) 10. Firebase Cloud Messaging (mobile apps) 11. Twilio SMS (critical alerts) 12. Mattermost (self-hosted)

6. Testing Strategy

6.1 Testing Philosophy

Goal: Achieve **100% test coverage** for all notification channels with: - Unit tests for individual channel logic - Integration tests with mock servers - E2E tests with real event flows - Performance tests for high-volume scenarios - Failure scenario tests

6.2 Unit Testing Strategy

Framework: Go's built-in testing package + testify assertions

Coverage Areas:

1. Channel Initialization

- Valid configuration
- Invalid configuration
- Missing required fields
- Enable/disable logic

2. Message Formatting

- Different notification types
- Priority levels
- Template rendering
- Special characters/escaping
- Unicode support

3. Payload Construction

- Slack webhook payload
- Telegram Bot API payload
- Discord webhook payload
- Email MIME formatting

4. Error Handling

- Network errors
- Authentication failures
- Invalid webhook URLs
- Malformed payloads
- Timeout scenarios

5. Configuration

- Loading from config.yaml
- Environment variable substitution
- Default values
- Validation logic

Example Test Structure:

```
func TestSlackChannel_Send(t *testing.T) {
    tests := []struct {
        name           string
        notification    *Notification
        wantErr         bool
        errContains     string
    }{
        {
            name: "success - info notification",
            notification: &Notification{
                Title:    "Test",
                Message: "Test message",
                Type:     NotificationTypeInfo,
            },
            wantErr: false,
        },
        {
            name: "error - channel disabled",
            notification: &Notification{
                Title: "Test",
            },
            wantErr:    true,
            errContains: "disabled",
        },
        // ... more test cases
    }

    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            // Test implementation
        })
    }
}
```

6.3 Integration Testing Strategy

Framework: httptest for mock servers + integration test suite

Mock Server Requirements:

1. Mock Slack Webhook Server

```
func mockSlackServer() *httptest.Server {  
    return httptest.NewServer(http.HandlerFunc(func(w  
        http.ResponseWriter, r *http.Request) {  
            // Verify payload structure  
            // Simulate various responses (200, 401, 500)  
            // Test rate limiting  
        })))  
}
```

2. Mock SMTP Server

- Use MailHog or similar SMTP mock
- Verify email headers, body, attachments
- Test authentication mechanisms

3. Mock Telegram Bot API Server

```
func mockTelegramServer() *httptest.Server {  
    // Implement sendMessage endpoint mock  
    // Test message formatting  
    // Test media uploads  
}
```

Test Scenarios:

1. Success Paths

- Message successfully sent
- Correct payload format
- Proper authentication

2. Failure Paths

- Network timeouts
- HTTP 4xx errors (auth, rate limit)
- HTTP 5xx errors (server errors)
- Invalid SSL certificates
- Connection refused

3. Retry Logic

- Verify retry on transient failures
- Exponential backoff validation
- Max retry limit enforcement

4. Rate Limiting

- Slack: 1 message/second
- Discord: 30 messages/minute

- Respect rate limits
- Queue management

6.4 E2E Testing Strategy

Framework: Full integration test suite

Test Flows:

1. Task Failure Flow

Task fails → Event emitted → Notification rule matches → Channels selected → Messages sent → Delivery confirmed

2. Worker Health Alert Flow

Worker health degrades → Event emitted → High-priority notification → Multiple channels (Slack + Email + Telegram) → Delivery confirmed

3. Workflow Completion Flow

Workflow completes → Event emitted → Template applied → Notification sent → Success logged

4. Multi-Channel Notification

- Send to Slack, Email, Telegram simultaneously
- Verify all channels receive notification
- Verify channel-specific formatting

5. Rule-Based Routing

- Different events trigger different rules
- Priority escalation
- Channel selection based on conditions

6.5 Performance Testing Strategy

Goals: - Handle 1000 notifications/minute - <100ms notification dispatch latency - Graceful degradation under load

Test Scenarios:

1. High Volume

- Send 1000 notifications concurrently
- Measure throughput
- Monitor resource usage

2. Rate Limiting

- Verify queuing under rate limits
- No dropped notifications
- Fair queue management

3. Concurrent Channels

- Send to 5 channels simultaneously
- Measure parallel processing
- Identify bottlenecks

Tools: - Go's testing/quick for property-based testing - Custom benchmarks with go test -bench - Load testing with vegeta or similar

6.6 Test Coverage Goals

Coverage Targets:

Component	Current Coverage	Target Coverage	Priority
Notification Engine Core	~30%	95%	HIGH
Slack Channel	0%	100%	HIGH
Telegram Channel	N/A	100%	HIGH
Email Channel	0%	100%	HIGH
Discord Channel	0%	90%	MEDIUM
Event Bus	0%	95%	HIGH
Configuration	~20%	90%	MEDIUM
Integration Tests	~10%	80%	HIGH

Overall Target: 95% coverage for notification system

7. Setup Guides for Integrations

7.1 Slack Integration Setup Guide

Prerequisites

- Slack workspace admin access
- Ability to create incoming webhooks

Step 1: Create Slack App

1. Go to <https://api.slack.com/apps>

2. Click “Create New App”
3. Select “From scratch”
4. Enter app name: “HelixCode Notifications”
5. Select your workspace

Step 2: Enable Incoming Webhooks

1. In your app settings, go to “Incoming Webhooks”
2. Toggle “Activate Incoming Webhooks” to **On**
3. Click “Add New Webhook to Workspace”
4. Select the channel (e.g., #helix-notifications)
5. Click “Allow”

Step 3: Copy Webhook URL

1. Copy the Webhook URL (looks like: `https://hooks.slack.com/services/T000000000/B000000000/XXXXXXXXXXXX`)

Step 4: Configure HelixCode

1. Add to `.env` file:

```
HELIX_SLACK_WEBHOOK_URL=https://hooks.slack.com/services/
YOUR/WEBHOOK/URL
```

2. Or add to `config.yaml`:

```
notifications:
  channels:
    slack:
      enabled: true
      webhook_url: ${HELIX_SLACK_WEBHOOK_URL}
      channel: "#helix-notifications"
      username: "HelixCode Bot"
```

Step 5: Test Integration

Using HelixCode CLI

```
helix notify test --channel slack --message "Test notification"
```

Or via API

```
curl -X POST http://localhost:8080/api/v1/notifications/test \
  -H "Content-Type: application/json" \
  -d '{
    "channel": "slack",
```

```
"title": "Test",  
"message": "Testing Slack integration",  
"type": "info"  
}'
```

Step 6: Configure Notification Rules

Edit config.yaml to define when to send Slack notifications:

```
notifications:  
  rules:  
    - name: "Critical Failures to Slack"  
      condition: "event.type == task_failed AND  
        event.data.priority == critical"  
      channels: ["slack"]  
      priority: urgent
```

Troubleshooting

- **Error: “invalid_token”** → Check webhook URL is correct
- **Error: “channel_not_found”** → Verify channel exists and bot has access
- **No notifications** → Check enabled: true in config
- **Rate limited** → Slack allows 1 message/second; check rate limit config

Advanced: Slack Block Kit (Future Enhancement)

For rich formatting with blocks: 1. Use Block Kit Builder: <https://api.slack.com/block-kit> 2. Configure in HelixCode (when implemented):
yaml slack:
use_blocks: true templates: task_failure: "path/to/block-template.json"

7.2 Telegram Integration Setup Guide

Prerequisites

- Telegram account
- Ability to create a bot via BotFather

Step 1: Create Telegram Bot

1. Open Telegram and search for @BotFather
2. Start a chat and send /newbot
3. Follow prompts:
 - Bot name: HelixCode Notifications

- Username: helixcode_bot (must end in _bot)

4. **Save the bot token** (looks like:

123456789:ABCdefGHIjklMNOpqrsTUVwxyz)

Step 2: Get Your Chat ID

Option A: Using userinfobot 1. Search for @userinfobot in Telegram 2. Start a chat 3. Copy your chat ID (e.g., 123456789)

Option B: Manual method 1. Send a message to your bot 2. Visit: https://api.telegram.org/bot<YOUR_BOT_TOKEN>/getUpdates 3. Find "chat": {"id":123456789} in JSON response

For group notifications: 1. Add bot to group 2. Send a message in group 3. Use getUpdates to find group chat ID (starts with -)

Step 3: Configure HelixCode

1. Add to .env file:

```
HELIX_TELEGRAM_BOT_TOKEN=123456789:ABCdefGHIjklMNOpqrsTUVwxyz
HELIX_TELEGRAM_CHAT_ID=123456789
```

2. Or add to config.yaml:

```
notifications:
  channels:
    telegram:
      enabled: true
      bot_token: ${HELIX_TELEGRAM_BOT_TOKEN}
      chat_id: ${HELIX_TELEGRAM_CHAT_ID}
```

Step 4: Test Integration

Using HelixCode CLI

```
helix notify test --channel telegram --message "Test
notification"
```

Or via API

```
curl -X POST http://localhost:8080/api/v1/notifications/test \
-H "Content-Type: application/json" \
-d '{
  "channel": "telegram",
  "title": "Test",
  "message": "Testing Telegram integration",
```

```
"type": "info"
}'
```

Step 5: Configure Notification Rules

```
notifications:
  rules:
    - name: "All failures to Telegram"
      condition: "event.type == task_failed"
      channels: ["telegram"]
      priority: high
```

Troubleshooting

- **Error: “Unauthorized”** → Check bot token is correct
- **Error: “Chat not found”** → Verify chat ID and that you’ve messaged the bot first
- **Error: “Bot was blocked by the user”** → Unblock the bot in Telegram
- **No notifications** → Ensure enabled: true in config

Advanced Features

Formatting: - Telegram supports HTML and Markdown - Configure in config.yaml: `yaml telegram: parse_mode: "HTML" # or "Markdown"`

Inline Keyboards (Future): - Add action buttons to notifications - Example: “Retry Task”, “View Logs”, “Dismiss”

Media Support (Future): - Send images, documents with notifications - Useful for charts, logs, screenshots

7.3 Email (SMTP) Integration Setup Guide

Prerequisites

- SMTP server access (Gmail, Office365, custom server)
- Email account credentials or app password

Step 1: Choose Email Provider

Option A: Gmail 1. Enable 2-factor authentication 2. Generate app password: - Go to Google Account → Security - Select “App passwords” - Generate password for “Mail” - Save the 16-character password

Option B: Office 365 1. Use your Office 365 credentials 2. Ensure SMTP is enabled for your account

Option C: Custom SMTP Server 1. Get SMTP server address and port 2. Obtain authentication credentials

Step 2: Configure HelixCode

For Gmail:

```
# .env file
HELIX_EMAIL_SMTP_SERVER=smtp.gmail.com
HELIX_EMAIL_SMTP_PORT=587
HELIX_EMAIL_USERNAME=your-email@gmail.com
HELIX_EMAIL_PASSWORD=your-app-password-here
HELIX_EMAIL_FROM=your-email@gmail.com
HELIX_EMAIL_RECIPIENTS=admin@example.com,team@example.com
```

For Office 365:

```
# .env file
HELIX_EMAIL_SMTP_SERVER=smtp.office365.com
HELIX_EMAIL_SMTP_PORT=587
HELIX_EMAIL_USERNAME=your-email@company.com
HELIX_EMAIL_PASSWORD=your-password
HELIX_EMAIL_FROM=your-email@company.com
HELIX_EMAIL_RECIPIENTS=admin@company.com
```

Config.yaml:

```
notifications:
  channels:
    email:
      enabled: true
      smtp:
        server: ${HELIX_EMAIL_SMTP_SERVER}
        port: 587
        username: ${HELIX_EMAIL_USERNAME}
        password: ${HELIX_EMAIL_PASSWORD}
        from: "HelixCode <${HELIX_EMAIL_FROM}>"
        tls: true
        auth_method: plain
      recipients:
        default: ["admin@example.com"]
        critical: ["admin@example.com", "oncall@example.com"]
```

Step 3: Test Integration

Using HelixCode CLI

```
helix notify test --channel email --message "Test email notification"
```

Or via API

```
curl -X POST http://localhost:8080/api/v1/notifications/test \
  -H "Content-Type: application/json" \
  -d '{
    "channel": "email",
    "title": "Test Email",
    "message": "Testing email integration from HelixCode",
    "type": "info",
    "metadata": {
      "recipients": ["admin@example.com"]
    }
  }'
```

Step 4: Configure Email Templates (Future Feature)

```
notifications:
  channels:
    email:
      templates:
        enabled: true
        html: true
        path: "/etc/helixcode/email-templates"
```

Troubleshooting

- **Error: “Authentication failed”** → Check username/password, ensure app password for Gmail
- **Error: “Connection refused”** → Verify SMTP server and port
- **Error: “TLS handshake failed”** → Check TLS settings, try port 465 (SSL) instead of 587 (TLS)
- **Error: “Recipient not specified”** → **Known bug, fix pending** - ensure `metadata.recipients` is set
- **Emails in spam** → Configure SPF/DKIM records for your domain

Common SMTP Ports

- **587** - TLS (recommended)
- **465** - SSL (legacy)

- 25 - Unencrypted (not recommended)

Rate Limiting

Most providers have rate limits: - Gmail: 500 emails/day (free), 2000/day (workspace) - Office 365: Varies by plan - Custom servers: Check with your admin

Configure rate limits:

```
email:
  rate_limit:
    max_per_hour: 100
```

7.4 Discord Integration Setup Guide

Prerequisites

- Discord server admin access
- Ability to manage webhooks

Step 1: Create Webhook

1. Open Discord and go to your server
2. Right-click the channel for notifications (e.g., #helix-alerts)
3. Select “Edit Channel”
4. Go to “Integrations” → “Webhooks”
5. Click “New Webhook”
6. Configure:
 - Name: HelixCode
 - Avatar: Upload HelixCode logo (optional)
7. Click “Copy Webhook URL”

Step 2: Configure HelixCode

```
# .env file
HELIX_DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/
123456789/abcdefghijklmnop

# config.yaml
notifications:
  channels:
    discord:
      enabled: true
      webhook_url: ${HELIX_DISCORD_WEBHOOK_URL}
```



```
retry:
  max_attempts: 3
rate_limit:
  max_per_minute: 30
```

Step 3: Test Integration

```
helix notify test --channel discord --message "Test Discord
notification"
```

Troubleshooting

- **Error: 401 Unauthorized** → Check webhook URL is correct
- **Error: 404 Not Found** → Webhook may have been deleted
- **Rate limited** → Discord allows 30 requests/minute per webhook
- **Messages not appearing** → Check channel permissions

Advanced: Discord Embeds (Future)

Rich embeds with colors, fields, thumbnails:

```
discord:
  use_embeds: true
  embed_color: "#FF6B00" # HelixCode brand color
```

8. Implementation Plan

8.1 Phase 1: Fix Critical Issues & Testing Infrastructure

Duration: 1 week **Priority:** CRITICAL

Tasks: 1. **Fix Email Channel Bug (BLOCKER)** - Fix recipient extraction from metadata (line 426) - Add proper recipient handling - Support multiple recipients -
Test case: Send email with recipient in metadata

1. Create Mock Server Infrastructure

- Implement mock Slack webhook server
- Implement mock Discord webhook server
- Implement mock SMTP server (or use MailHog)
- Create helper functions for integration tests

2. Add Basic Unit Tests

- Slack channel tests (100% coverage target)
- Email channel tests (100% coverage target)
- Discord channel tests (100% coverage target)

- Engine core tests (enhance existing)

Deliverables: - Email channel working correctly - Mock server infrastructure ready - Unit test suite with 80%+ coverage - CI/CD integration for tests

Success Metrics: - All existing channels tested and functional - Zero critical bugs - Test coverage >80% for notification module

8.2 Phase 2: Telegram Integration & Configuration

Duration: 1 week **Priority:** HIGH

Tasks: 1. **Implement Telegram Channel** ``go type TelegramChannel struct { name string enabled bool botToken string chatID string }

func (c *TelegramChannel*) Send(ctx context.Context, notification Notification) error { // Implement Telegram Bot API sendMessage // Support HTML/Markdown formatting // Handle media attachments (future) } ``

1. Add Telegram Unit Tests

- Send message success
- Authentication failures
- Invalid chat ID
- Rate limiting
- Message formatting (HTML/Markdown)

2. Add Telegram Integration Tests

- Mock Telegram Bot API server
- Test full request/response cycle
- Test error scenarios

3. Update Configuration System

- Add notification section to config.yaml
- Add environment variable support
- Add configuration validation
- Add config hot-reload (future)

4. Enhance Slack Integration

- Add Slack Block Kit support
- Add rich formatting
- Add attachment support
- Test new features

Deliverables: - Telegram integration fully functional - Telegram test coverage 100% - Configuration management enhanced - Slack enhanced with rich formatting

Success Metrics: - Telegram notifications working end-to-end - Configuration loaded from config.yaml - Enhanced Slack notifications with blocks

8.3 Phase 3: Event-Driven Hook System

Duration: 2 weeks **Priority:** HIGH

Tasks: 1. **Design Event Bus Architecture** ```go type EventBus struct { subscribers map[string][]EventHandler mutex sync.RWMutex }

func (bus EventBus) Subscribe(eventType string, handler EventHandler) func (bus EventBus) Publish(event Event) ```

1. Implement Event Bus

- Subscriber management
- Event publishing
- Async event handling
- Error handling
- Event filtering

2. Integrate Event Bus with Components

- Task Manager: Emit events on task status changes
- Workflow Executor: Emit workflow events
- Worker Pool: Emit worker events
- API Handlers: Emit API events

3. Connect Notification Engine to Event Bus

- Subscribe to relevant events
- Map events to notification rules
- Apply rule conditions
- Trigger notifications

4. Implement Notification Rules Engine

- Parse rule conditions
- Evaluate conditions against events
- Select channels based on rules
- Apply priority escalation
- Template application

5. Add Rule Configuration

- Load rules from config.yaml
- Validate rule syntax
- Dynamic rule updates
- Rule testing utility

6. Testing

- Event bus unit tests
- Integration tests: event → notification
- E2E tests: task failure → Slack notification

- Performance tests: 1000 events/second

Deliverables: - Event bus implemented and tested - All components emitting events - Notifications triggered automatically - Rule engine functional

Success Metrics: - Task failures automatically send notifications - Worker issues trigger alerts - Workflow completions notify users - Zero manual notification invocations

8.4 Phase 4: Retry Logic & Reliability

Duration: 1 week **Priority:** MEDIUM

Tasks: 1. **Implement Retry Mechanism** ``go type RetryConfig struct { MaxAttempts int InitialDelay time.Duration MaxDelay time.Duration BackoffMultiplier float64 }

func (c *SlackChannel*) *SendWithRetry*(ctx context.Context, notification Notification) error { // Exponential backoff retry logic } ``

1. Add Rate Limiting

- Token bucket algorithm
- Per-channel rate limits
- Queue management
- Backpressure handling

2. Implement Notification Queue

- Persistent queue (Redis-backed)
- Delivery guarantees
- Failed notification handling
- Dead letter queue

3. Add Observability

- Metrics: notifications sent/failed
- Logging: structured logs
- Tracing: distributed tracing
- Dashboards: Grafana integration

4. Testing

- Retry logic tests
- Rate limiting tests
- Queue persistence tests
- Failure scenario tests

Deliverables: - Retry mechanism implemented - Rate limiting enforced - Notification queue operational - Observability added

Success Metrics: - 99.9% notification delivery rate - No lost notifications - Graceful degradation under load - Clear visibility into notification status

8.5 Phase 5: Additional Integrations

Duration: 2 weeks **Priority:** MEDIUM

Tier 1 Integrations: 1. **Generic Webhooks** - Configurable HTTP method - Custom headers - Payload templates - Authentication options

1. Microsoft Teams

- Workflows API integration
- Adaptive Cards support
- Action buttons

Tier 2 Integrations: 3. **PagerDuty** - Events API v2 - Incident creation - Alert deduplication

1. GitHub Issues

- Issue creation from failures
- Comment addition
- Label management

Testing: - Unit tests for each integration - Integration tests with mocks - E2E tests with real APIs (optional)

Deliverables: - Generic webhooks functional - Microsoft Teams integration - PagerDuty integration (optional) - GitHub integration (optional)

8.6 Phase 6: Documentation & Website

Duration: 1 week **Priority:** MEDIUM

Tasks: 1. **Create Setup Guides** - Slack setup guide (see Section 7.1) - Telegram setup guide (see Section 7.2) - Email setup guide (see Section 7.3) - Discord setup guide (see Section 7.4) - Generic webhook setup guide - Microsoft Teams setup guide

1. Update Website (Github-Pages-Website)

- Add “Integrations” section to homepage
- Create integrations showcase
- Add visual integration logos
- Link to setup guides

Additions to index.html:

```
<!-- New Integrations Section -->
<section id="integrations" class="integrations">
  <div class="container">
    <div class="section-header">
      <h2 class="section-title">Powerful Integrations</h2>
      <p class="section-subtitle">Connect HelixCode with your favorite tools</p>
    </div>
    <div class="integrations-grid">
      <div class="integration-card">
        <div class="integration-logo">
          
        </div>
        <h3>Slack</h3>
        <p>Real-time notifications with rich formatting</p>
        <a href="docs/integrations/slack.html" class="btn btn-sm">Setup Guide</a>
      </div>
      <!-- More integration cards... -->
    </div>
  </div>
</section>
```

2. API Documentation

- REST API for notifications
- Webhook payload formats
- Authentication
- Rate limits

3. Configuration Reference

- Complete config.yaml reference 
- Environment variables 
- Rule syntax guide
- Template guide

Deliverables: - All setup guides complete - Website updated with integrations - API documentation published - Configuration reference available

8.7 Phase 7: Performance & Scale Testing

Duration: 1 week **Priority:** LOW

Tasks: 1. **Load Testing** - 1000 notifications/minute - Concurrent channel sending - Memory/CPU profiling

1. Optimization

- Connection pooling
- Batch sending (where applicable)
- Caching

2. Benchmarking

- Establish performance baselines
- Regression testing

Deliverables: - Load tests passing - Performance optimizations - Benchmark suite

9. Testing Implementation Details

9.1 Test File Structure

```
helix_code/
├── internal/
│   └── notification/
│       ├── engine.go
│       ├── engine_test.go (existing - enhance)
│       ├── slack_test.go (NEW)
│       ├── telegram_test.go (NEW)
│       ├── email_test.go (NEW)
│       ├── discord_test.go (NEW)
│       ├── webhooks_test.go (NEW)
│       └── testutil/
│           ├── mock_servers.go (NEW)
│           ├── test_helpers.go (NEW)
│           └── fixtures.go (NEW)
├── test/
│   ├── integration/
│   │   ├── notification_integration_test.go (enhance existing)
│   │   ├── slack_integration_test.go (NEW)
│   │   ├── telegram_integration_test.go (NEW)
│   │   └── email_integration_test.go (NEW)
│   └── e2e/
│       └── notification_e2e_test.go (NEW)
```

9.2 Sample Test: Slack Channel

File: internal/notification/slack_test.go

```
package notification
```

```
import (  
    "context"  
    "encoding/json"  
    "io"  
    "net/http"  
    "net/http/httptest"  
    "testing"  
    "github.com/stretchr/testify/assert"  
    "github.com/stretchr/testify/require"  
)
```

```
func TestSlackChannel_New(t *testing.T) {  
    tests := []struct {  
        name      string  
        webhook   string  
        channel   string  
        username  string  
        wantEnabled bool  
    }{  
        {  
            name:      "valid configuration",  
            webhook:  "https://hooks.slack.com/services/T/B/X",  
            channel:  "#helix",  
            username: "bot",  
            wantEnabled: true,  
        },  
        {  
            name:      "empty webhook",  
            webhook:  "",  
            channel:  "#helix",  
            username: "bot",  
            wantEnabled: false,  
        },  
    }  
  
    for _, tt := range tests {  
        t.Run(tt.name, func(t *testing.T) {
```



```

        channel := NewSlackChannel(tt.webhook, tt.channel,
tt.username)
        assert.Equal(t, tt.wantEnabled, channel.IsEnabled())
        assert.Equal(t, "slack", channel.GetName())
    })
}

}

func TestSlackChannel_Send(t *testing.T) {
    tests := []struct {
        name            string
        notification     *Notification
        serverStatus    int
        serverCheck      func(*testing.T, *http.Request)
        wantErr          bool
        errContains      string
    }{
        {
            name: "success - info notification",
            notification: &Notification{
                Title:    "Test Title",
                Message: "Test Message",
                Type:     NotificationTypeInfo,
            },
            serverStatus: http.StatusOK,
            serverCheck: func(t *testing.T, r *http.Request) {
                // Verify payload
                var payload map[string]interface{}
                body, _ := io.ReadAll(r.Body)
                err := json.Unmarshal(body, &payload)
                require.NoError(t, err)

                assert.Equal(t, "#helix", payload["channel"])
                assert.Contains(t, payload["text"], "Test Title")
                assert.Equal(t, ":information_source:",
payload["icon_emoji"])
            },
            wantErr: false,
        },
        {
            name: "error - server error",
            notification: &Notification{
                Title: "Test",
                Message: "Test",
            },

```

```

        Type: NotificationTypeError,
    },
    serverStatus: http.StatusInternalServerError,
    wantErr: true,
    errContains: "status 500",
},
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        // Setup mock server
        server := httptest.NewServer(http.HandlerFunc(func(w
http.ResponseWriter, r *http.Request) {
            if tt.serverCheck != nil {
                tt.serverCheck(t, r)
            }
            w.WriteHeader(tt.serverStatus)
        }))
        defer server.Close()

        // Create channel with mock server URL
        channel := NewSlackChannel(server.URL, "#helix",
"bot")

        // Send notification
        err := channel.Send(context.Background(),
tt.notification)

        if tt.wantErr {
            require.Error(t, err)
            if tt.errContains != "" {
                assert.Contains(t, err.Error(),
tt.errContains)
            }
        } else {
            require.NoError(t, err)
        }
    })
}

func TestSlackChannel_IconForType(t *testing.T) {
    channel := &SlackChannel{}

    tests := []struct {

```

```

    notifType NotificationType
    wantIcon  string
}{
    {NotificationTypeSuccess, ":white_check_mark:"},
    {NotificationTypeWarning, ":warning:"},
    {NotificationTypeError, ":x:"},
    {NotificationTypeAlert, ":rotating_light:"},
    {NotificationTypeInfo, ":information_source:"},
}

for _, tt := range tests {
    t.Run(string(tt.notifType), func(t *testing.T) {
        icon := channel.GetIconForType(tt.notifType)
        assert.Equal(t, tt.wantIcon, icon)
    })
}

func TestSlackChannel_RateLimit(t *testing.T) {
    // Test that rate limiting is respected
    // Slack allows 1 message/second

    requestCount := 0
    server := httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
            requestCount++
            w.WriteHeader(http.StatusOK)
        }))
    defer server.Close()

    channel := NewSlackChannel(server.URL, "#test", "bot")

    // Send 5 notifications quickly
    for i := 0; i < 5; i++ {
        notif := &Notification{
            Title: "Test",
            Message: "Test",
            Type: NotificationTypeInfo,
        }
        _ = channel.Send(context.Background(), notif)
    }

    // With rate limiting, this should take ~4 seconds
    // Without rate limiting, all would be sent immediately

```

```
    // TODO: Implement rate limiting first, then this test will
        validate it
}
```

9.3 Sample Test: Email Channel

File: internal/notification/email_test.go

```
package notification
```

```
import (
    "context"
    "testing"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)
```

```
func TestEmailChannel_RecipientExtraction(t *testing.T) {
    // Test for the bug fix in line 426

    channel := NewEmailChannel("smtp.example.com", 587, "user",
        "pass", "from@example.com")

    notification := &Notification{
        Title: "Test",
        Message: "Test message",
        Type: NotificationTypeInfo,
        Metadata: map[string]interface{}{
            "recipients": []string{"admin@example.com",
                "team@example.com"},
        },
    }

    // This should NOT return "no recipient specified" error
    // TODO: Implement fix first
    // err := channel.Send(context.Background(), notification)
    // assert.NoError(t, err)
}
```

```
func TestEmailChannel_MultipleRecipients(t *testing.T) {
    // Test sending to multiple recipients
    // TODO: Implement with mock SMTP server
}
```

```
func TestEmailChannel_HTMLFormatting(t *testing.T) {
```

```

    // Test HTML email formatting (future feature)
    // TODO: Implement HTML support first
}

```

9.4 Integration Test: Mock Servers

File: internal/notification/testutil/mock_servers.go

```

package testutil

import (
    "encoding/json"
    "io"
    "net/http"
    "net/http/httptest"
    "sync"
)

type MockSlackServer struct {
    *httptest.Server
    Requests []SlackRequest
    mutex     sync.Mutex
}

type SlackRequest struct {
    Channel  string
    Username string
    Text     string
    IconEmoji string
}

func NewMockSlackServer() *MockSlackServer {
    mock := &MockSlackServer{
        Requests: make([]SlackRequest, 0),
    }

    mock.Server = httptest.NewServer(http.HandlerFunc(func(w
        http.ResponseWriter, r *http.Request) {
        mock.mutex.Lock()
        defer mock.mutex.Unlock()

        // Parse request
        var req SlackRequest
        body, _ := io.ReadAll(r.Body)
        json.Unmarshal(body, &req)

```

```

        mock.Requests = append(mock.Requests, req)

        w.WriteHeader(http.StatusOK)
    )))

    return mock
}

func (m *MockSlackServer) GetRequests() []SlackRequest {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    return m.Requests
}

func (m *MockSlackServer) Reset() {
    m.mutex.Lock()
    defer m.mutex.Unlock()
    m.Requests = make([]SlackRequest, 0)
}

```

10. Final Recommendations

10.1 Critical Priorities (Do First)

1. **Fix Email Bug** - Blocker for production use
2. **Add Tests** - Critical for reliability
3. **Implement Telegram** - High user demand
4. **Event System** - Core functionality for hooks

10.2 Implementation Order

Week 1-2: Phase 1 (Fix bugs + Testing infrastructure)
 Week 3-4: Phase 2 (Telegram + Config + Enhance Slack)
 Week 5-6: Phase 3 (Event-driven system)
 Week 7: Phase 4 (Retry + Reliability)
 Week 8-9: Phase 5 (Additional integrations)
 Week 10: Phase 6 (Documentation + Website)
 Week 11: Phase 7 (Performance testing)

Total Duration: ~11 weeks for complete implementation

10.3 Resource Requirements

Developer Time: - 1 senior Go developer (full-time) - 1 QA engineer (part-time for testing strategy) - 1 technical writer (part-time for documentation)

Infrastructure: - Test SMTP server (MailHog or similar) - Test Telegram bot - Test Slack workspace - CI/CD pipeline updates

10.4 Success Metrics

Code Quality: - 95%+ test coverage for notification system - Zero critical bugs - All linters passing

Functionality: - 100% automated event triggering - Slack, Telegram, Email, Discord fully functional - Generic webhooks working - Rule-based routing operational

Performance: - Handle 1000 notifications/minute - <100ms notification dispatch - 99.9% delivery rate

Documentation: - Setup guides for all integrations - API documentation complete - Configuration reference available - Website updated

11. Conclusion

The HelixCode notification and hook integration system has a solid foundation with good architectural patterns, but requires significant implementation work to reach production readiness.

Current State: - Partial implementations of Slack, Email, Discord - Good engine framework - **Critical gaps:** Telegram missing, email broken, zero tests, no event system

Recommended Actions: 1. Immediate: Fix email bug 2. Short-term: Add comprehensive tests 3. Medium-term: Implement Telegram + event system 4. Long-term: Additional integrations + performance optimization

Expected Outcome: After full implementation, HelixCode will have a world-class notification system with: - Automated event-driven notifications - 100% test coverage - Multiple integration options - Enterprise-grade reliability - Comprehensive documentation

This will significantly enhance HelixCode's value proposition and user experience.

Report End